# Recap — archiving summaries in a database

Recap turns an uploaded meeting recording into a structured recap: a short overview, decisions, action items with owners, and a topic outline with timestamps.

The database is how those recaps stay available after the browser tab closes. A visitor can list past meetings, open one again, export it to PDF or Markdown, and search across what was decided. The original recording is temporary. Processing finishes, then the file is deleted unless the visitor explicitly keeps it.

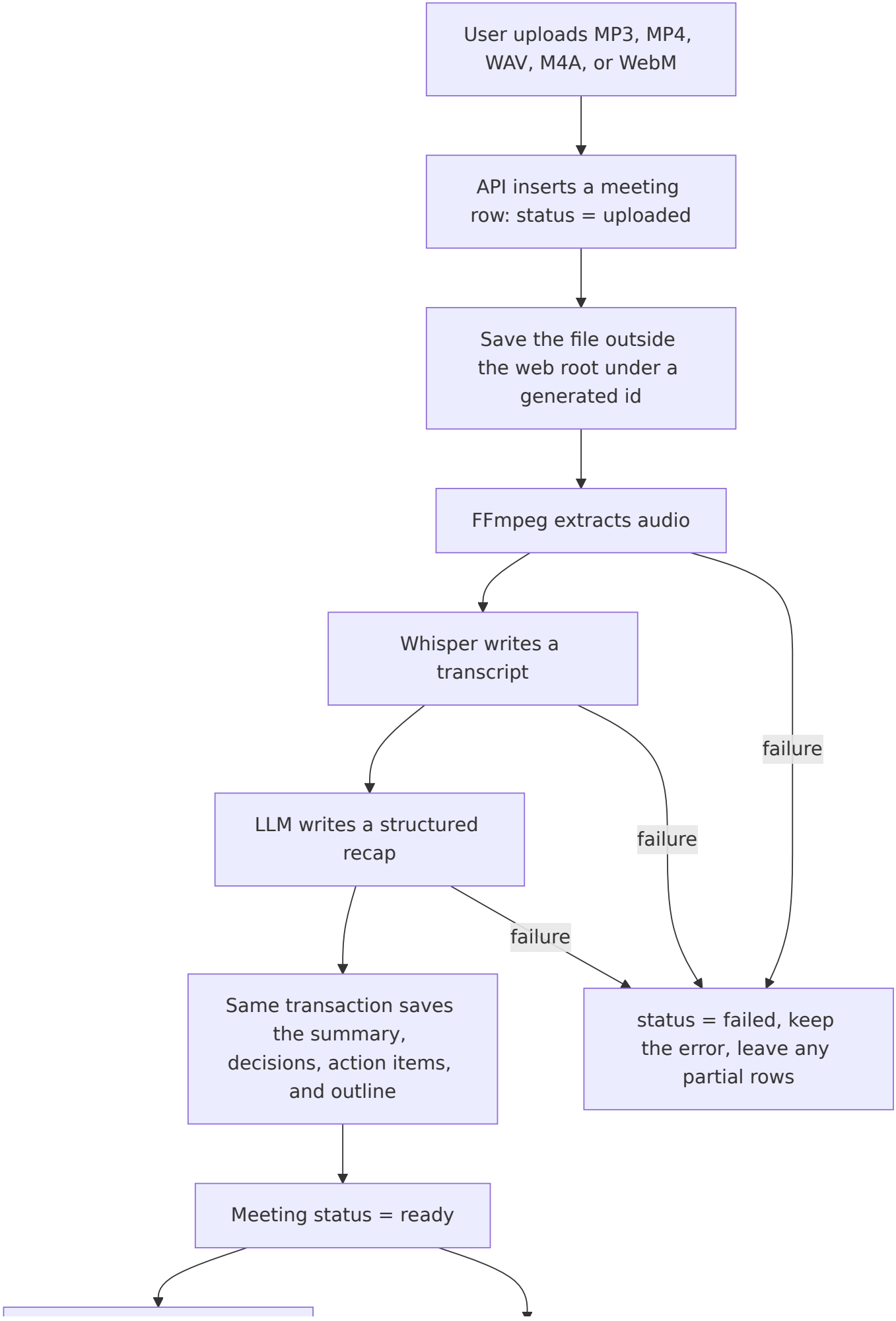
There is no login. The public site still keeps a separate archive per browser. The first visit creates an `owners` row and sets a long-lived cookie. Every meeting uploaded from that browser stores that row's `owner_id`.

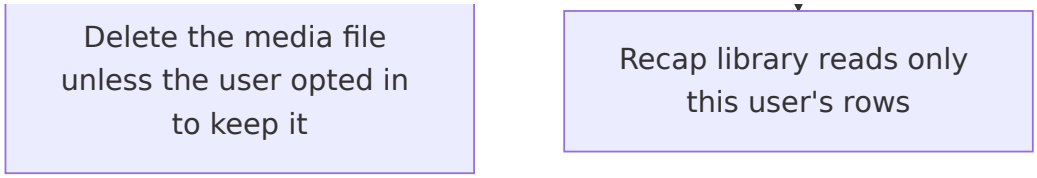
This fits the proposed stack: React for the library UI, FastAPI for the pipeline, Whisper and an LLM for the content, and **SQLite** as the archive.

Author: Amanda Xaysouthep · mentor note for capstone team (Fall 2026 – Spring 2027).

---

## Pipeline


User uploads MP3, MP4, WAV, M4A, or WebM
API inserts a meeting row: status = uploaded
Save the file outside the web root under a generated id
FFmpeg extracts audio
Whisper writes a transcript
LLM writes a structured recap
Same transaction saves the summary, decisions, action items, and outline
Meeting status = ready
status = failed, keep the error, leave any partial rows
failure
failure
failure




Each box that touches the database writes through the API with parameterized queries. The React app never opens the database file itself.

Status values on `meetings.status`:

| Status | Meaning |
|---|---|
| `uploaded` | File accepted, row created, pipeline not started |
| `extracting` | FFmpeg is pulling audio out of video |
| `transcribing` | Whisper is running |
| `summarizing` | The LLM is structuring the transcript |
| `ready` | Current summary is saved and safe to show |
| `failed` | Pipeline stopped; `error_message` is a short generic reason |

The UI polls `GET /meetings/{id}` (or listens on a small status endpoint) and renders the recap only when status is `ready`.

---

# What is kept

| Artifact | Kept by default? | Where |
|---|---|---|
| Recording (audio/video) | No. Deleted after a successful run unless `keep_media` is true | Disk, outside the web root. The database stores a path and a flag, never the bytes |
| Transcript | Yes, attached to the meeting | `transcripts` and optional `transcript_segments` |
| Structured summary | Yes. This is the archive | `summaries` plus child tables |

| | | |
|---|---|---|
| Earlier summary versions | Yes, when the user re-runs summarization | New `summaries` row; one row is marked current |

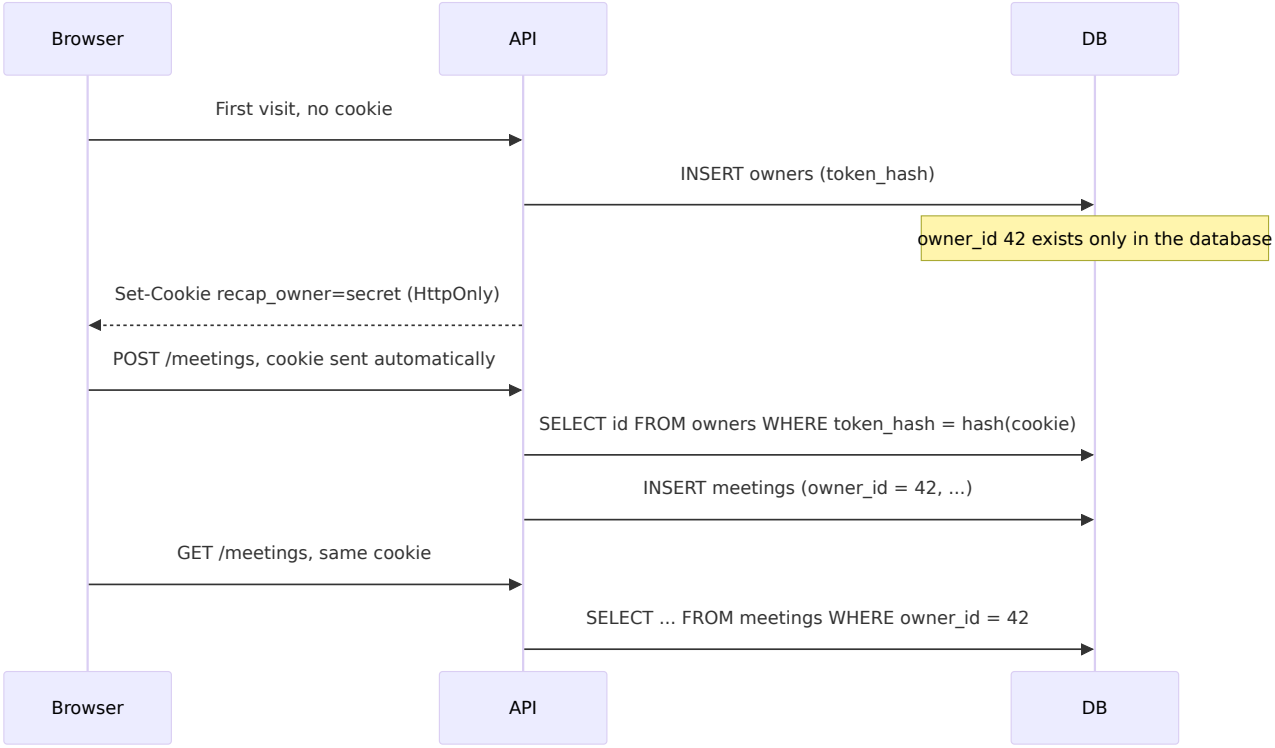
Keeping the transcript means a later model or prompt can rebuild the recap without asking the user to upload the recording again. Keeping summary versions means a re-run does not erase the recap they already exported.

Meeting content stays tied to the browser that uploaded it. Every read and delete checks `meetings.owner_id` against the owner resolved from the cookie.

---

# How `owner_id` is tracked

`owner_id` is an integer primary key on `owners`. It never goes in the cookie, the URL, or the page. The browser holds a random secret; the database holds the id.




Step by step:

1. A request arrives with no `recap_owner` cookie. The API generates a 256-bit secret with a cryptographic random generator.
2. It stores only the SHA-256 hash of that secret on a new `owners` row. The raw secret is not written to the database.

3. The response sets `recap_owner` to the raw secret. The cookie is `HttpOnly`, `Secure`, and `SameSite=Lax`, with a long lifetime (about a year) so closing the tab keeps the archive.
4. The browser sends that cookie on later requests to this site. JavaScript cannot read it.
5. The API hashes the cookie, loads `owners.id`, and uses that integer as `owner_id` on insert and in every `WHERE owner_id = ?` clause.

A second visitor, or the same person in another browser, gets a new `owners` row and a different cookie. Their library query returns only meetings stamped with their own `owner_id`. Meeting summaries are not public and are not searchable across visitors.

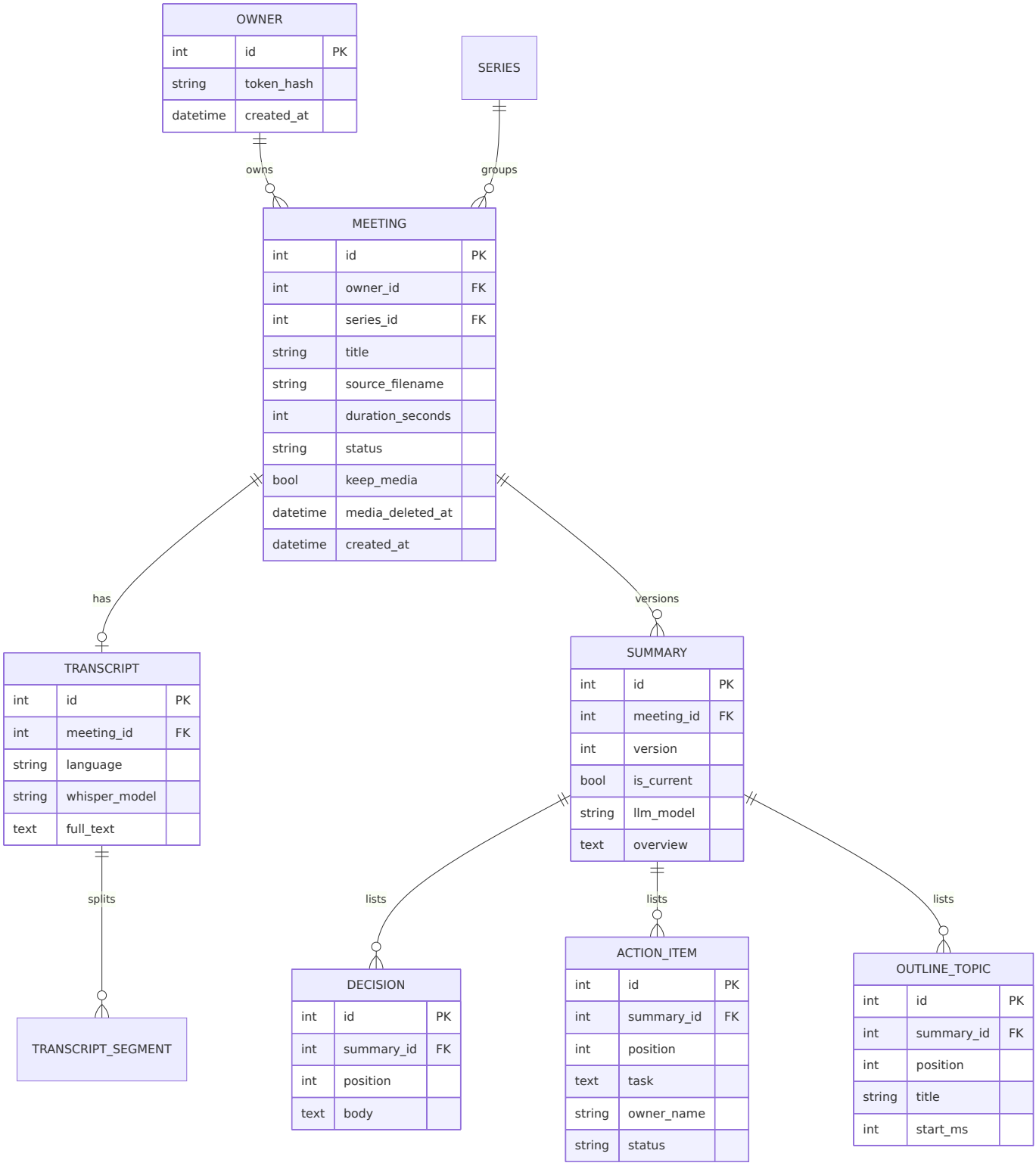
The integer id itself stays off the cookie because sequential ids are easy to guess. Possession of the secret is what proves this browser owns those rows.

Clearing site data, or switching browsers, starts a new empty archive. The old rows remain in the database and stay unreachable without the original cookie. That is the tradeoff of skipping accounts. An optional recovery code can be the same secret shown once in the UI, for someone who wants to paste it into another browser. That code is still not a username or password.

`owner_name` on an action item is unrelated. It is a name heard in the meeting (“Priya”), not this browser’s `owner_id`.

---

# Tables




`SERIES` is optional and can wait until the stretch goal (compare a recurring meeting). `owner_name` on an action item is a name heard in the meeting (“Priya”), not the browser owner.

## `owners`

One row per browser archive. No email and no password.

| Column | Notes |
| --- | --- |
| `id` | Primary key. This value is `meetings.owner_id`. It is never sent to the browser |
| `token_hash` | SHA-256 of the cookie secret. Unique. The raw secret is not stored |
| `created_at` | UTC |

## `meetings`

One row per upload. This is the card in the recap library.

| Column | Notes |
| --- | --- |
| `id` | Primary key |
| `owner_id` | Foreign key → `owners.id`, copied from the cookie lookup at upload time |
| `series_id` | Nullable. Filled in only if the team builds recurring-meeting comparison |
| `title` | Editable. Default: filename, then replace with a title the LLM suggests |
| `source_filename` | Original name for display only, after stripping any directory (`recording.mp4`) |
| `duration_seconds` | From FFmpeg, once known |
| `status` | See the status table above |
| `error_message` | Short text safe to show the user. No file paths, tokens, or transcript excerpts |
| `keep_media` | Default `false` |
| `media_path` | Nullable path on disk while the file exists |
| `media_deleted_at` | Set when the file is removed |

| | |
|---|---|
| `created_at`, `updated_at` | UTC |

Index: `(owner_id, created_at DESC)` so "my recaps, newest first" stays cheap.

## `transcripts`

One transcript per meeting.

| Column | Notes |
|---|---|
| `id` | Primary key |
| `meeting_id` | Unique foreign key → `meetings.id` |
| `language` | Whisper language code, e.g. `en` |
| `whisper_model` | `base`, `small`, … so a later run can be compared |
| `full_text` | Plain text of the meeting |
| `created_at` | UTC |

## `transcript_segments`

Optional in the fall. Worth adding once the outline needs real timestamps, and required if speaker labels show up in the spring.

| Column | Notes |
|---|---|
| `id` | Primary key |
| `transcript_id` | Foreign key |
| `start_ms`, `end_ms` | Offset from the start of the recording |
| `speaker_label` | Nullable. `A`, `B`, or a name the user assigns later |
| `text` | That slice of speech |
| `position` | Order inside the transcript |

## `summaries`

One meeting can have several summaries. Re-running the LLM inserts a new version and clears `is_current` on the previous one, inside one transaction.

| Column | Notes |
|---|---|
| `id` | Primary key |
| `meeting_id` | Foreign key → `meetings.id` |
| `version` | 1, 2, 3… unique per meeting |
| `is_current` | Exactly one `true` per meeting once status is `ready` |
| `llm_model` | Model id used for this version |
| `prompt_version` | Short label such as `recap-v1`, so prompt changes are traceable |
| `overview` | The two-paragraph summary |
| `payload_json` | The full model JSON, kept so export matches what the model returned |
| `created_at` | UTC |

`payload_json` is the snapshot. The child tables below are the queryable copy, written in the same transaction from that JSON. If they ever disagree, rebuild the child rows from `payload_json`.

Partial unique index: one current summary per meeting (`meeting_id` where `is_current = 1`).

## `decisions`

| Column | Notes |
|---|---|
| `id` | Primary key |
| `summary_id` | Foreign key → `summaries.id` |
| `position` | Display order |
| `body` | One decision, one row |

## `action_items`

| Column | Notes |
| --- | --- |
| `id` | Primary key |
| `summary_id` | Foreign key |
| `position` | Display order |
| `task` | What needs to happen |
| `owner_name` | Nullable name from the transcript |
| `due_hint` | Nullable free text (“Friday”, “before the demo”) |
| `status` | `open` or `done`. Default `open`. Lets the library show unfinished follow-ups |

### `outline_topics`

| Column | Notes |
| --- | --- |
| `id` | Primary key |
| `summary_id` | Foreign key |
| `position` | Chronological order |
| `title` | Topic label |
| `start_ms` | Where to jump in the recording |
| `notes` | Optional one-line detail |

Delete a meeting and the database removes its transcript, segments, summaries, decisions, action items, and outline rows (`ON DELETE CASCADE`). If `keep_media` was set, the API deletes the file in the same user action, then deletes the row.

---

## How a finished recap is written

The LLM is asked for JSON in this shape:

```
{
  "title": "Sprint planning — Oct 14",
  "overview": "The team committed to shipping the upload flow in
December...",
  "decisions": [
    "December demo is the upload-to-summary loop only"
  ],
  "action_items": [
    { "task": "Wire Whisper into the API", "owner": "Alex", "due_hint":
"next week" }
  ],
  "outline": [
    { "title": "Scope for December", "start_ms": 125000, "notes":
"Agreed to skip speaker labels" }
  ]
}
```

FastAPI validates that JSON (required keys, string lengths, list sizes) before any insert. A response that fails validation sets `meetings.status = failed` and does not publish a half summary.

Then, in one transaction:

1. Insert `summaries` with `version = previous + 1`, `is_current = true`, and the raw JSON in `payload_json`.
2. Set the previous current summary to `is_current = false`.
3. Insert `decisions`, `action_items`, and `outline_topics` from the arrays.
4. Update `meetings.title` if the user has not edited it, and set `status = ready`.

After the transaction commits, delete the media file when `keep_media` is false and set `media_deleted_at`.

---

## What the archive can answer

These are ordinary SQL queries. The bound `owner_id` is the `owners.id` looked up from the cookie hash on that request.

**Library, newest first**

```
SELECT id, title, status, created_at
FROM meetings
WHERE owner_id = ?
ORDER BY created_at DESC;
```

**The recap on screen** — current summary plus its three lists, ordered by `position`.

**Search across archived recaps.** Add an FTS5 virtual table over overview, decision body, action task, and outline title. A query like "budget" returns meetings this user owns, with a snippet, without scanning raw media.

**Open follow-ups**, once action items have a status:

```
SELECT m.title, a.task, a.owner_name
FROM action_items AS a
JOIN summaries AS s ON s.id = a.summary_id AND s.is_current = 1
JOIN meetings AS m ON m.id = s.meeting_id
WHERE m.owner_id = ?
  AND a.status = 'open'
ORDER BY m.created_at DESC;
```

**Recurring series** (spring stretch): meetings that share `series_id`, compared on their current decisions and still-open action items.

Export to Markdown or PDF reads the same current summary. It does not call the LLM again.

---

# API surface

There is no login route. If the cookie is missing, the API creates an `owners` row and sets the cookie before it handles the request.

| Method | Path | Database effect |
|---|---|---|
| `POST` | `/meetings` | Resolve `owner_id` from the cookie, insert `meetings` (`uploaded`), store the file, start the pipeline |

| | | |
|---|---|---|
| `GET` | `/meetings` | List meetings for that `owner_id` only |
| `GET` | `/meetings/{id}` | Meeting, current summary, decisions, actions, outline. 404 if the id is not theirs |
| `PATCH` | `/meetings/{id}` | Rename, or toggle an action item `open` / `done` |
| `POST` | `/meetings/{id}/summarize` | New summary version from the stored transcript |
| `GET` | `/meetings/{id}/export?format=md` | Render the current summary. `pdf` uses the same rows |
| `DELETE` | `/meetings/{id}` | Delete the row (cascade) and any kept media file |

Unknown ids and another browser’ s meeting ids return the same not-found response, so the API does not reveal that a meeting exists.

---

# SQLite for the capstone

SQLite matches the constraint that this runs on student laptops: one file ( `recap.db` ), no extra server, backups are a file copy, and the schema is real SQL the team can show in a demo.

Use it with:

- Foreign keys turned on ( `PRAGMA foreign_keys = ON` )
- WAL mode so a status poll can read while a summary is being written
- Parameterized statements only ( `?` placeholders). No string-built SQL
- The database file and the media directory outside the React app and outside any static-file route

Postgres is the same tables and the same queries if the May demo is multi-user. Moving is a connection string and a migration tool, not a new design.

Suggested layout on disk:

```
data/
  recap.db
  media/
    7f3c...a1.wav    # generated name, removed after processing by
default
```

---

## Fall vs spring

**Fall (the archive exists).** `owners`, `meetings`, `transcripts`, `summaries`, and the three child tables. The first visit sets the cookie. Upload stamps `owner_id` on the meeting. The UI lists past recaps for that cookie and opens one. Media deletion runs by default. Export reads the database.

**Spring (the archive gets more useful).** Full-text search. Action-item status. Segment timestamps precise enough to jump back into a kept recording. Optional `series_id` if they compare a recurring meeting. Speaker labels on `transcript_segments` if diarization lands.

A December demo can already show the point of the database: process a recording, close the app, reopen it, and the recap is still there — while the media file is gone.

---

## Guardrails this design keeps

- Test recordings are used only with consent from the people in them.
- The browser that holds the cookie is the only reader of those meetings, transcripts, and summaries. Another visitor’ s cookie resolves to a different `owner_id`.
- The cookie secret is stored only as a hash. API keys for a hosted LLM stay in the environment, not in a table.
- Logs record meeting id and status changes. They do not record transcript text, summary text, or filenames that contain personal names.
- Uploaded names are stripped to a base filename. Files are written under `data/media/` with a generated name, and the resolved path must stay inside that directory.
- A failed LLM response is rejected. The previous current summary, if any, stays current.